

Design System MD.

A design system built for agents to **route through**, not read through. One always-loaded page holds the map and the rules. Everything verbose loads on demand.

FORMAT

One main file. Routed references.
A gated pipeline for everything new.

AUDIENCE

Whoever owns the project,
and the Claude working with them.

EDITION

v1.0 · June 2026
Internal draft

THE PROBLEM

One file holding the whole system means every request pays for all of it.

Tokens, forty component specs, pattern recipes, voice rules, all loaded so the agent can answer a question about one button. At best that's wasted tokens on every single request. At worst the agent skims, and the one rule that mattered gets missed.

THE SOLUTION

The main file is a **router plus a constitution**: roughly one page, always loaded. It holds a decision map and the rules needed on every task. Everything verbose sits behind it and loads only when routed to.

The decision map is written in **task language, not taxonomy language**. Not a table of contents. Triage:

```
Building a form?      → patterns/forms.md + components/core/text-field.md
Page-level states?   → patterns/screen-states.md
Need exact values?   → references/tokens.md
Using a component?    → read its revisions; build the current version.
New component / token → STOP. Read the creation gate first.
```

WHY SEGMENTS FOLLOW ACCESS PATTERNS, NOT TAXONOMY

LOOKUP

Tokens

Exact values. Small, dense, needed often.

SELECTIVE

Components

One or two per task. Never all. Split per component.

CONDITIONAL

Patterns

Only when the task matches the recipe.

AMBIENT

Rules & voice

Cheap, applies everywhere. Lives in the main file.

THE RULE OF ADMISSION

Only rules needed on **every** task earn a place in the main file. Anything conditional goes behind a routing key. No exceptions. This is the discipline that keeps the always-loaded cost at one page.

One always-loaded file. Three routed territories.

```

design-system/
├─ design-system.md      ← always loaded. The router + the rules.
├─ components/
│   ├─ core/             ← the official set. Stable. One file per component.
│   └─ extensions/       ← project-born components. Delta files only.
├─ patterns/            ← recipes: forms, screen-states, navigation, tables...
├─ references/
│   ├─ tokens.md         ← color, type, spacing, elevation, motion.
│   └─ voice.md          ← copy rules, terminology, error message style.

```

WHAT EACH PIECE OWNS

LOCATION	OWNS
design-system.md	The decision map, operating rules, creation gate, both floors, the extensions index. ~100 lines. The only file every request loads.
components/core/	The official component set. Canonical and stable, never edited mid-project .
components/extensions/	Everything born in projects. Each file is a delta on a core component, or a gated net-new one.
patterns/	How components compose to solve recurring problems. Forms, page-level states, confirmation flows.
references/	Dense lookups (tokens) and direction (voice). The component-file format itself lives in the main file, next to the states table it depends on.

WHY EXTENSIONS LIVE HERE, NOT IN HANDOFF FOLDERS

A component born inside one outcome's handoff folder is invisible to the next project, so it gets reinvented, and now there are three appointment pickers. One searchable surface; status tags carry the maturity signal.

Seven sections. ~100 lines. The agent's fixed cost per request.

Half of it routes, half of it governs. Each section is compact because the verbose version of everything lives behind the map.

01 Decision map

Task-language routing. "Building a form? → load..." Tells the agent which references to pull for this task, and nothing else.

02 Operating rules

Precedence (production system beats prototype; doc beats prototype on rules), naming conventions, and what to do when no component fits: **flag it, never invent it**.

03 Creation gate

The hard stop. The agent cannot create or modify anything canonical (components, tokens, patterns) on its own. Full text on the next page.

04 Status glossary

Three words, plain English. **Draft**: new, not yet proven in real use; may still change. **Proven**: used in real screens here and held up; safe to reuse. **Retired**: superseded; don't use, kept so old work still resolves.

05 Minimum Viable States table

The state floor: the smallest set of states per component class (and per page) without which the thing is undesignated. Page 7.

06 Minimum Viable Components list

The width floor: the smallest component set that makes the system usable at all. Page 8.

07 Extensions index

One line per extension: name, status, what it extends, when to use it. The agent checks this before ever reporting "nothing fits".

WHAT THE MAIN FILE IS NOT

Not a summary of the system. Not a table of contents. It's the smallest set of text that lets the agent **route correctly and behave correctly**. Every spec, recipe, and token table stays behind it. If a section grows past its job, it's absorbing content that belongs in a routed file.

Claude composes from the system. Only humans change the system.

Composition from registered components and tokens is always allowed. **Anything canonical is gated**: components, tokens, and patterns change only when the user explicitly commissions it.

THE GATE TEXT: COPY THIS INTO THE MAIN FILE

```
## Creation gate
You may NOT create, modify, or extend anything canonical on your
own: components, tokens, or patterns.
- Every element in your output maps to a registered component,
  or is composed purely from registered ones.
- Every visual value maps to a token. No hardcoded values.

If nothing fits:
1. STOP. Do not improvise an unregistered component,
  token, or pattern.
2. Name the gap: what's needed, why nothing fits, the closest
  existing component or token, the delta required.
3. Wait. Only an explicit instruction from the user,
  "yes, create it" / "extend X", lets you proceed.

When told to proceed: write the file in the component-file
format below (delta-only for extensions, status: draft), and
add one line to the extensions index.
```

THE COMPONENT-FILE FORMAT

What a new file must contain. It lives in the main file, beside the Minimum Viable States table it draws its states from.

```
name: slug
status: draft      draft | proven | retired
version: 1
extends: parent | none

## Anatomy      parts & structure
## States       every state for its class (MVS)
## Variants     if any
## Rules        must / must-not, invisible logic
## Revisions    what changed, + re-check on breaking
```

TWO RULES THAT BIND IT

- **Extensions are delta-only.** List only what differs from the parent; everything unmentioned inherits. Net-new files carry the full set.
- **States are not optional.** The file is not done until it covers every state the Minimum Viable States table requires for its class. That table is the completeness check.

WHY THE GATE EXISTS

EFFECT

WITHOUT THE GATE

The human is the author of record Claude generates components; the project owner degrades into auditing a stream of unreviewed files.

The gap report is a decision point Auto-creation paves the cowpath instantly. Often the right answer is restructuring the design to use what exists; the gate is the moment that call gets made.

The system grows at human pace Components and tokens multiply at Claude-speed until routing itself gets expensive. A token edit is worse: it silently propagates everywhere.

Never mutate core. Write deltas with lineage.

Core files are canonical and stable. Everything new is a separate file that declares what it extends and contains **only what differs**. Everything unmentioned inherits from the base: small files, no copy-paste drift.

THE EXTENSION FILE

```
name: select-grouped
status: draft
version: 1
extends: select      ← or "none" if net-new
born-in: ora-event-creation
owner: eman
delta: option grouping with
      sticky group headers

## What differs from core/select.md
(states, anatomy, rules: delta only)
```

THE LIFECYCLE

STATUS MEANS

draft	New. Built but not yet proven in real use. May still change.
proven	Used in real screens here and held up. Safe to reuse.
retired	Superseded or dropped. Don't use; kept so old work resolves.

Status is self-set, not granted. The project owner marks an extension *proven* once it has been used in real screens and held up, and may move a proven one into `core/` when it has earned a place. No approval step, no gatekeeper.

EXPANDING AN EXISTING COMPONENT: ADDITIVE VS. BREAKING

CHANGE	MECHANISM	PERMISSION
Additive	New variant, size, or state → extension file, <code>extends: x</code> . Ship today.	None needed. Status <i>draft</i> is the permission.
Breaking	Changes how the base behaves for everyone. Write it as a scoped override, declared in <code>handoff.md</code> under Decisions, so the change is visible rather than silent.	Scoped, declared, traceable.

ONE CASCADE EVERYWHERE

core → extension → outcome-level override (declared in `handoff.md`)
Same conflict-resolution shape as the `handoff` format's "doc wins on rules". One model, no special cases.

A change to a component is recorded in the component, so it travels.

When a canonical thing changes, the change is written into its own file as a revision. Claude reads the current revision every time it touches that component, so it always builds to the latest truth. **The revision is how a change reaches every place the component is used.**

THE REVISION BLOCK, INSIDE THE COMPONENT FILE

```
name: button
version: 4

## Revisions
- r4: radius 8 → 4px.
  Breaking — re-check any layout that
  leaned on the larger radius.
- r3: added `loading` state.
  Additive — existing usages unaffected.
- r2: focus ring uses --focus token.
  Additive.
```

Two fields per entry: **what changed**, and on a breaking change, **what to re-check**. That second line is a flag for attention, never an automatic action.

HOW A CHANGE PROPAGATES

- **By reading.** Claude touches a screen using the button, reads `button.md`, sees r4, and builds the current spec. No stale copy survives contact.
- **As work happens.** Each place that uses the button comes up to r4 the next time it is rendered or edited. Propagation rides on real work.
- **On request, for a full sweep.** "Bring the project to button r4" is a user instruction — a commission, same shape as the gate. Claude can update every instance at once, but only when asked.

THE ONE BOUNDARY: NO SILENT SWEEPS

Reading a revision updates what Claude is **already working on**. It never triggers an unrequested rewrite across the codebase — a one-line note must not become a forty-file find-and-replace. The full sweep is always a separate, explicit user instruction.

ADDITIVE VS. BREAKING — SAME DISTINCTION, APPLIED TO CHANGES

KIND	WHAT THE REVISION SAYS	EFFECT ON EXISTING USAGES
Additive	New state, variant, or token added. Nothing existing changes meaning.	None. They keep working as-is.
Breaking	An existing value or behavior changes. Entry names what to re-check.	Flagged for attention, surfaced — never resolved silently.

BY CLASS

- **Core & extensions:** revisions in-file, as above. An extension also notes whether it **inherits** a parent's revision or pins to the old behavior — so a button change doesn't silently rewrite button-icon.
- **Patterns:** same block, but breaking entries usually read *"re-check"*, not a mechanical swap — a pattern is a composition, so a change needs judgment, not a find-and-replace.

Per class: the states without which it is undesigned.

Classes inherit downward: Input means everything Interactive requires, plus its own. Pages are a class too: a screen can pass every component check and still have no answer for "what does this page look like before data exists."

CLASS	EXAMPLES	REQUIRED STATES
Static	badge, divider, card, avatar	default only, explicitly no others
Interactive	button, link, tab, menu item	default · hover · focus · active · disabled
Input	text field, select, checkbox, toggle	Interactive + empty · filled · error · read-only
Overlay	modal, drawer, popover, toast	open · closing / dismiss affordance · overflow
Async / data	table, list, dashboard widget	populated · loading · empty · error · partial
AI surface	chat output, summary, suggestion	idle · working (steps visible) · streaming · complete · error · refusal · interrupted
Page / screen	every screen archetype	loaded · first-load (skeleton) · first-use empty · no-results empty · page error · partial failure · no-access

All classes, cross-cutting: visible keyboard focus · mobile-width behavior. State names are canonical: each gets a one-line definition under the table, so one designer's "skeleton" is never another's "loading".

THE DISTINCTIONS THAT GET MISSED

- **First-use empty ≠ no-results empty.** Zero data needs onboarding energy and a primary action. An empty filter result needs "no matches, clear filters". Same slot, opposite messages.
- **Refusal is not an error.** Its own visual shape; merge them and users read refusals as bugs.
- **Empty vs. filled inputs.** Designers ship the filled state; the dev's Claude invents placeholder behavior.
- **Partial failure.** Five widgets load, one errors. Defined answer, or you get a full-page error for a 20% failure.
- **Interrupted.** User stops generation mid-stream. What does half-finished output look like?

ONE TABLE, FIVE CONSUMERS

Authoring bar for new components · the readiness check before a component is marked proven · the agent's completeness check ("table has no empty state: gap or intentional?") · the source of the prototype's state toolbar · the pre-handoff QA list.

The smallest set that makes composition possible.

The creation gate only works if hitting it is rare. So the core set is sized by a coverage test: **the common screen archetypes (form, list, detail, dashboard, AI interaction) must be assemblable by composition alone.**

CLASS	CORE SET (~21 COMPONENTS)
Static	card · badge/tag · divider
Interactive	button (primary / secondary / ghost / icon) · link · tabs
Input	text field · textarea · select · checkbox · radio · toggle · field scaffold (label + helper + error wrapper)
Overlay	modal · dropdown menu · tooltip · toast
Async / data	table · list · pagination · loading (spinner + skeleton) · empty state · alert/banner
Layout	app shell (header + content + sidebar)

CONDITIONAL MODULE: AI SURFACES

Required core **only if the product has AI features**: prompt input · streaming text block · AI working/steps indicator · source/citation chip · refusal block. These are the components the handoff format's behavior rules keep referencing; making them core gives those rules a place to live instead of being re-specified per handoff.

TWO DELIBERATE CALLS

- **Field scaffold is a component, not a pattern.** Label position, required-asterisk, helper, error placement: one canonical wrapper, or every form re-litigates it.
- **Empty state is a component, not just a state.** The states table requires having one; this gives a standard one to use.

DELIBERATELY EXCLUDED

Date picker, file upload, rich text editor, wizard/stepper, data viz, breadcrumb. All legitimate, but heavy or archetype-specific. They're the **expected first extensions**, born through the gate when a project needs them.

THE VIABILITY DEFINITION: WHEN A DESIGN SYSTEM MD COUNTS AS ONE

- (1) tokens exist for color, type, spacing
- (2) every core component exists with all Minimum Viable States for its class
- (3) the main file carries the map, operating rules, gate, and glossary. A checkable, binary bar, and a scopeable kickoff phase for clients who arrive with no system.

The handoff asks four questions. This system supplies the machinery behind two of them.

Design → Dev Handoff v1.0 defines *what travels* between designer and developer. Design System MD defines *what both sides resolve against*.

IN THE HANDOFF FORMAT

BACKED BY

"What are all the states this can be in?" (the prototype's state toolbar)	Generated from the Minimum Viable States page row . "Every state reachable by URL" now has a canonical list of which states that means.
"What in the prototype should I ignore?" (the Not Decisions section)	The registry settles it: anything not a registered component or token is a default to override, not a decision.
"Intentional vs. Claude default?"	The creation gate guarantees no legitimate component exists without a registered file: the registry is the ground truth . Unregistered = default.
The five-minute pre-handoff check	One added line: <i>force every state in the page row</i> , drawn from the Minimum Viable States table.
"When prototype and doc disagree"	One cascade everywhere: core → extension → outcome override, doc wins on rules.

THE ONE DEPENDENCY, NOW PRICED

The handoff format assumed a mature production design system without defining one. This document closes that gap with a number on it: **viability = tokens + ~21 core components × their Minimum Viable States + the one-page main file**. For a client arriving with nothing, that's a scopeable, quotable kickoff phase, not a vague prerequisite.

THE ONE BOUNDARY

Every project stands up its own Design MD. The system makes no claim about who owns it or what their title is. The only line that holds everywhere: **Claude composes from the system and never changes canon without the person's explicit say-so.**

What good looks like.

DESIGN-SYSTEM.MD (EXCERPT)

```
## Decision map
Form or inputs?      → patterns/forms.md
                     + core/text-field.md + core/select.md

Table or list?       → core/table.md + patterns/screen-states.md
Page-level states?   → patterns/screen-states.md
Exact values?        → references/tokens.md
AI output UI?        → core/ai/ (streaming, citations, refusal)
New component / token? → STOP. Creation gate below.
```

```
## Operating rules
- Production design system beats prototype. Doc beats
  prototype on rules. Never resolve conflicts silently.
- No component fits? Check the extensions index. Still
  nothing? Flag the gap. Never invent.
```

```
## Status glossary
draft: new, not yet proven. may change.
proven: used here and held up. reuse ok.
retired: superseded. kept for old refs.
```

```
## Extensions index
- select-grouped (draft, extends select)
  · grouped options w/ sticky headers. born: ora.
- appointment-picker (proven, net-new)
  · clinical scheduling. born: fch.
```

EXTENSIONS/SELECT-GROUPED.MD

```
name: select-grouped
status: draft
version: 2
extends: select
born-in: ora-event-creation
owner: eman
delta: option grouping with sticky
      group headers
```

```
## Revisions
- r2: group header sticky on scroll.
  Additive.
- r1: initial grouped variant.
```

```
## What differs from core/select.md
```

```
Anatomy
+ group header row: label, non-
  interactive, sticky on scroll
```

```
States (adds to inherited set)
+ group-collapsed
+ group-empty: header hidden when
  a group has zero options
```

```
Rules
- Groups sort by config order,
  never alphabetically.
- Keyboard: ↑↓ skips headers;
  typeahead matches options only.
```

```
Everything not listed inherits
from core/select.md.
```

The whole file is the delta. Once *select-grouped* has been used in real screens and held up, status flips to **proven**. If it earns a permanent place and meets the Input-class Minimum Viable States bar, it moves into core.

THE TEST, SAME AS THE HANDOFF'S

The system works when the agent can answer four questions without asking: **Where do I look for this task?** · **Does a component exist for this?** · **How much can I trust it?** · **What am I forbidden to invent?** All four answers live in one always-loaded page.